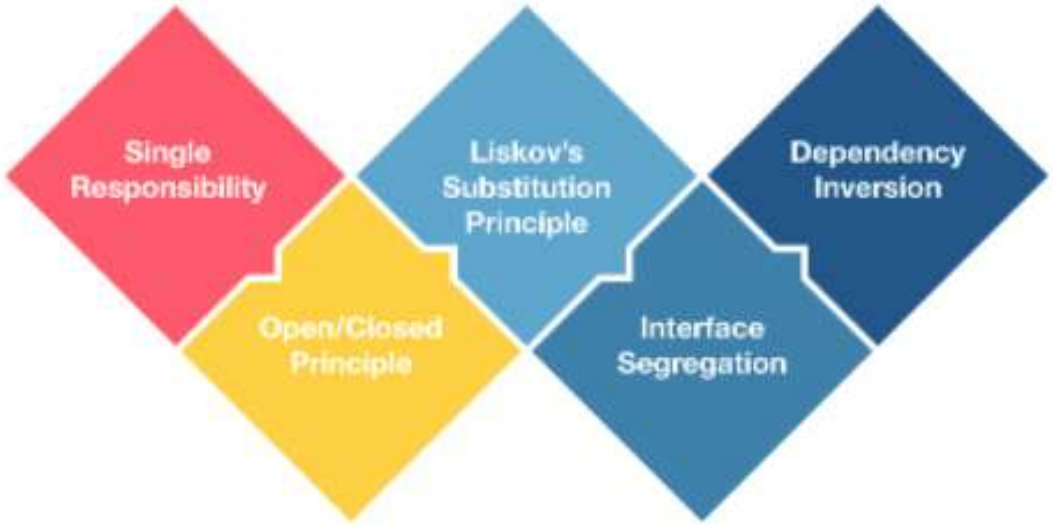
S.O.L.I.D.


Single Responsibility
Open/Closed Principle
Liskov's Substitution Principle
Interface Segregation
Dependency Inversion

Here are **SOLID principles in one-line interview answers**:

S (SRP) → A class should have **only one responsibility**.

O (OCP) → Software should be **open for extension, closed for modification**.

L (LSP) → Derived classes should **replace base classes without breaking behavior**.

I (ISP) → Clients should **not depend on unused interfaces**.

D (DIP) → Depend on **abstractions, not concrete implementations**.

Simple Way to Remember (Interview Tip)

S → One class = One job

O → Extend, don’t modify

L → Replace without breaking

I → Small, specific interfaces

D → Depend on interfaces

S — Single Responsibility Principle (SRP)

Definition: A class should have only **one reason to change**.

✅ Good Example

```
public class Invoice
{
public void CalculateTotal() { }
}

public class InvoicePrinter
{
public void PrintInvoice(Invoice invoice) { }
}
```

O — Open/Closed Principle (OCP)

Definition: Open for **extension**, closed for **modification**.

✅ Good Example

```
public abstract class Customer
{
public abstract double GetDiscount();
}

public class RegularCustomer : Customer
{
public override double GetDiscount() => 10;
}

public class PremiumCustomer : Customer
{
public override double GetDiscount() => 20;
}
```

L — Liskov Substitution Principle (LSP)

Definition: Derived classes should be replaceable for base classes **without breaking behavior**.

✅ Good Example

```
interface IPayment
{
void Pay(decimal amount);
}
class CreditCardPayment : IPayment
{
public void Pay(decimal amount)
{
Console.WriteLine($"Paid {amount} using Credit Card");
}
}

class UPIPayment : IPayment
{
public void Pay(decimal amount)
{
Console.WriteLine($"Paid {amount} using UPI");
}
}
```

```
class PaymentProcessor
{
public void ProcessPayment(IPayment payment, decimal amount)
{
payment.Pay(amount);
}
}
var processor = new PaymentProcessor();

IPayment payment1 = new CreditCardPayment();
processor.ProcessPayment(payment1, 1000);

IPayment payment2 = new UPIPayment();
processor.ProcessPayment(payment2, 500);
```

I — Interface Segregation Principle (ISP)

Definition: Don’t force classes to implement methods they don’t use.

✅ Good Example

```
public interface IWorkable
{
void Work();
}

public interface IEatable
{
void Eat();
}
public class Human : IWorkable, IEatable
{
public void Work() => Console.WriteLine("Human working");
public void Eat() => Console.WriteLine("Human eating");
}

public class Robot : IWorkable
{
public void Work() => Console.WriteLine("Robot working");
}
```

D — Dependency Inversion Principle (DIP)

Definition: Depend on **abstractions**, not on concrete classes.

✅ **Good Example**

```
public interface IMessageService
{
void Send();
}

public class EmailService : IMessageService
{
public void Send() { }
}

public class Notification
{
private readonly IMessageService _messageService;

public Notification(IMessageService messageService)
{
_messageService = messageService;
}

public void Notify()
{
_messageService.Send();
}
}
```